CSci 21 Data Structures and Algorithms
Module 4

Stacks

Learning Outcomes

LO 4.1 Implement accurate stack operations

LO 4.2 Implement accurate solutions for problems using stack ADT

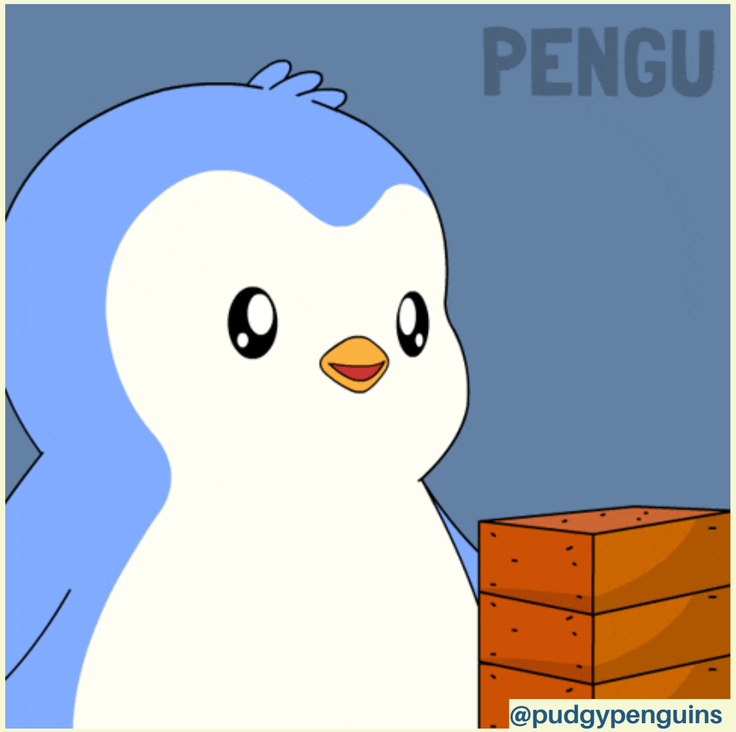
LO 4.3 Diagnose stack-specific errors

Setting the Expectations

PREREQUISITES

- Utilized **arrays** and **linked lists** in programming solutions.
- Utilized **pointers** in **C/C++.**

What's a Stack?



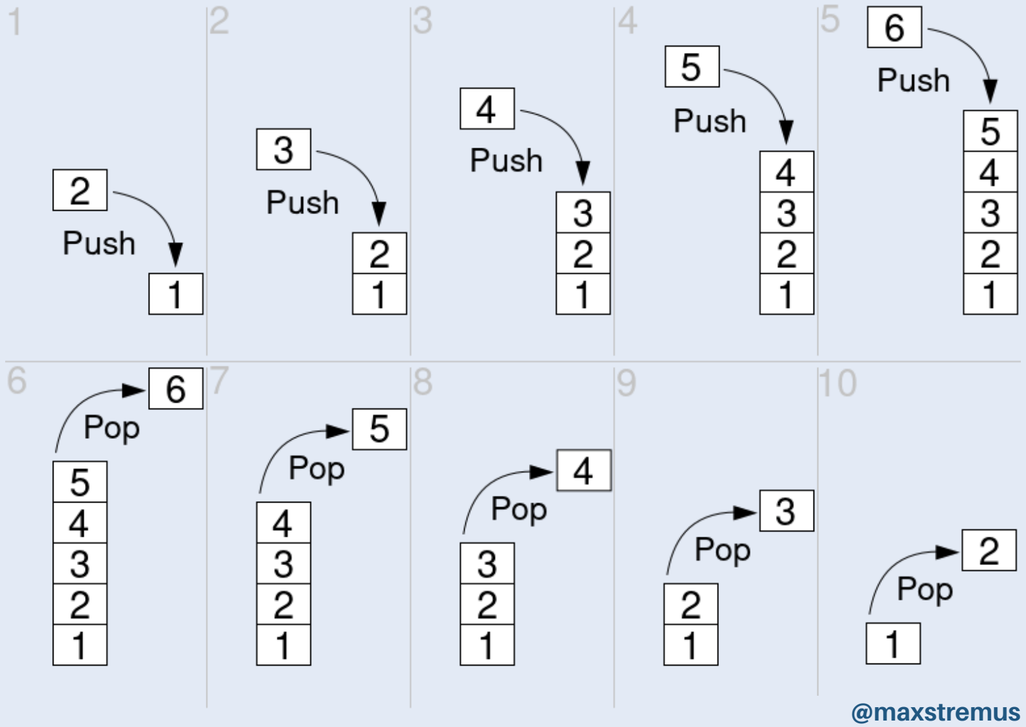
Last-in First-Out (LIFO)
linear data structure

Types of Stack

Bounded stack

Unbounded stack

Main Operations



Auxiliary Operations

Peek returns the element at the top of the stack

IsEmpty returns true if the stack is empty and false otherwise

IsFull returns true if the stack is full and false otherwise

Implement **Stack**

Solve Problems Involving **Stack**

Complexity of Stack Operations

OPERATION	TIME COMPLEXITY	WHY?
push	O(1)	array = make top the end, linked list = insert at the head
pop	O(1)	array = decrement the size, linked list = delete the head node
peek	O(1)	direct access at the end of the array or head of the list
isEmpty	O(1)	ask if size = 0 or head is null
isFull	O(1)	ask if size = capacity

Stack-specific Errors

1. **Stack Underflow**
   - ❌ Calling pop() or peek() on an empty stack → Crash!
   - ✅ Always check isEmpty() first.

2. **Stack Overflow (Bounded stack)**
   - ❌ Pushing onto a full stack
   - ✅ Check size == capacity before push.

3. **Incorrect Top Management**
   - ❌ In array stack: arr[++size] on push (wrong order!).
   - ✅ Push: arr[size++] = data. Pop: return arr[--size].

"The key to wisdom is constant and frequent questioning, for by doubting we are led to question, by questioning we arrive at the truth..."

- Peter Abelard

OPEN FORUM Q/A

for Module 4